

Module 3: Data Organization and Manipulation using Collections

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai campus

premanand.s@vit.ac.in

September 15, 2025

In module 3,

- Data Handling Strategies Using Lists, Tuples, Dictionary and Set
- Data Comprehension
- Data Modification
- Grouping and Categorization
- Iterators
- Data Selection - Ordered, Unordered and Unique Data
- Searching Techniques: Linear Search, Binary Search
- Sorting Techniques: Bubble Sort, Selection Sort- Insertion Sort- Quick Sort- Merge Sort.

What is Sorting?

- Sorting means arranging items in order.
- Example:
 - Unsorted: [7, 2, 9, 1, 5]
 - Sorted (Ascending): [1, 2, 5, 7, 9]
 - Sorted (Descending): [9, 7, 5, 2, 1]

Why Sorting?

- To search faster (Binary Search works only on sorted lists).
- To make data organized & readable.
- Examples,
 - Arranging marks in ascending order.
 - Sorting names in alphabetical order.
 - Organizing files by date.

Types - Sorting

- Bubble sort
- Selection sort
- Insertion sort
- Quick sort
- Merge sort

Bubble sort

- Imagine you are blowing bubbles - Bigger bubble goes down - Smaller bubble goes up.
- We compare two numbers at a time and swap if needed.

Bubble Sort Trace (Part 1)

Start: [5, 3, 8, 4, 2]

Pass 0

j=0: (5,3) $\rightarrow$ swap $\Rightarrow$ [3, 5, 8, 4, 2]

j=1: (5,8) $\rightarrow$ no swap $\Rightarrow$ [3, 5, 8, 4, 2]

j=2: (8,4) $\rightarrow$ swap $\Rightarrow$ [3, 5, 4, 8, 2]

j=3: (8,2) $\rightarrow$ swap $\Rightarrow$ [3, 5, 4, 2, 8]

Pass 1

j=0: (3,5) $\rightarrow$ no swap $\Rightarrow$ [3, 5, 4, 2, 8]

j=1: (5,4) $\rightarrow$ swap $\Rightarrow$ [3, 4, 5, 2, 8]

j=2: (5,2) $\rightarrow$ swap $\Rightarrow$ [3, 4, 2, 5, 8]

j=3: (5,8) $\rightarrow$ no swap $\Rightarrow$ [3, 4, 2, 5, 8]

Bubble Sort Trace (Part 2)

Pass 2	j=0: (3,4) $\rightarrow$ no swap $\Rightarrow$ [3, 4, 2, 5, 8]
	j=1: (4,2) $\rightarrow$ swap $\Rightarrow$ [3, 2, 4, 5, 8]
	j=2: (4,5) $\rightarrow$ no swap $\Rightarrow$ [3, 2, 4, 5, 8]
	j=3: (5,8) $\rightarrow$ no swap $\Rightarrow$ [3, 2, 4, 5, 8]

Pass 3	j=0: (3,2) $\rightarrow$ swap $\Rightarrow$ [2, 3, 4, 5, 8]
	j=1: (3,4) $\rightarrow$ no swap
	j=2: (4,5) $\rightarrow$ no swap
	j=3: (5,8) $\rightarrow$ no swap

Pass 4	Already sorted, no changes.
---------------	-----------------------------

Final Sorted List: [2, 3, 4, 5, 8]

Simple Bubble Sorting

Example (Syntax)

```
numbers = [5, 3, 8, 4, 2]

for i in range(len(numbers)):
    for j in range(len(numbers) - 1):
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] =
                numbers[j + 1], numbers[j]

print("Sorted List:", numbers)
```

Pass 1 and Pass 2

Initial List: [5, 3, 8, 4, 2]

Pass 1

Compare 5 & 3 → swap → [3, 5, 8, 4, 2]

Compare 5 & 8 → no swap → [3, 5, 8, 4, 2]

Compare 8 & 4 → swap → [3, 5, 4, 8, 2]

Compare 8 & 2 → swap → [3, 5, 4, 2, 8]

⇒ End of Pass 1: [3, 5, 4, 2, 8]

Pass 2

Compare 3 & 5 → no swap → [3, 5, 4, 2, 8]

Compare 5 & 4 → swap → [3, 4, 5, 2, 8]

Compare 5 & 2 → swap → [3, 4, 2, 5, 8]

Compare 5 & 8 → no swap → [3, 4, 2, 5, 8]

⇒ End of Pass 2: [3, 4, 2, 5, 8]

Pass 3, Pass 4, Pass 5

Pass 3

Compare 3 & 4 → no swap → [3, 4, 2, 5, 8]

Compare 4 & 2 → swap → [3, 2, 4, 5, 8]

Compare 4 & 5 → no swap → [3, 2, 4, 5, 8]

Compare 5 & 8 → no swap → [3, 2, 4, 5, 8]

⇒ End of Pass 3: [3, 2, 4, 5, 8]

Pass 4

Compare 3 & 2 → swap → [2, 3, 4, 5, 8]

Compare 3 & 4 → no swap → [2, 3, 4, 5, 8]

Compare 4 & 5 → no swap → [2, 3, 4, 5, 8]

Compare 5 & 8 → no swap → [2, 3, 4, 5, 8]

⇒ End of Pass 4: [2, 3, 4, 5, 8]

Pass 5

No swaps at all → **stop early!**

Bubble Sorting - Stop Early if Already Sorted

Example (Syntax)

```
numbers = [5, 3, 8, 4, 2]

for i in range(len(numbers)):
    swapped = False
    for j in range(len(numbers) - 1):
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] =
                numbers[j + 1], numbers[j]
            swapped = True
    if not swapped:
        break

print("Sorted List:", numbers)
```

Pass 1 and Pass 2

Initial List: [5, 3, 8, 4, 2]

Pass 1 ($i = 0, j = 0..3$)

Compare 5 & 3 → swap → [3, 5, 8, 4, 2]

Compare 5 & 8 → no swap → [3, 5, 8, 4, 2]

Compare 8 & 4 → swap → [3, 5, 4, 8, 2]

Compare 8 & 2 → swap → [3, 5, 4, 2, 8]

⇒ End of Pass 1: [3, 5, 4, 2, 8]

Pass 2 ($i = 1, j = 0..2$)

Compare 3 & 5 → no swap → [3, 5, 4, 2, 8]

Compare 5 & 4 → swap → [3, 4, 5, 2, 8]

Compare 5 & 2 → swap → [3, 4, 2, 5, 8]

⇒ End of Pass 2: [3, 4, 2, 5, 8]

Pass 3, Pass 4, Pass 5

Pass 3 ($i = 2, j = 0..1$)

Compare 3 & 4 $\rightarrow$ no swap $\rightarrow$ [3, 4, 2, 5, 8]

Compare 4 & 2 $\rightarrow$ swap $\rightarrow$ [3, 2, 4, 5, 8]

$\Rightarrow$ End of Pass 3: [3, 2, 4, 5, 8]

Pass 4 ($i = 3, j = 0$)

Compare 3 & 2 $\rightarrow$ swap $\rightarrow$ [2, 3, 4, 5, 8]

$\Rightarrow$ End of Pass 4: [2, 3, 4, 5, 8]

Pass 5 ($i = 4$)

No comparisons (inner loop empty).

$\Rightarrow$ Final Sorted List: [2, 3, 4, 5, 8]

Bubble Sorting - Reduce Comparisons Each Pass

Example (Syntax)

```
numbers = [5, 3, 8, 4, 2]

n = len(numbers)
for i in range(n):
    for j in range(n - i - 1):
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] =
                numbers[j + 1], numbers[j]

print("Sorted List:", numbers)
```

- Best case (already sorted): very few checks → fast
- Worst case (reverse order): lots of checks → slow
- Time Complexity: $O(n^2)$ (like a square table → rows $\times$ columns).

Questions to practice,

- Sort [9, 7, 5, 3, 1] using bubble sort.
- What happens if the list is already sorted — does bubble sort still work?
- Try sorting names instead of numbers: ["cat", "apple", "dog", "bat"].
- Modify bubble sort so it sorts in descending order.
- Why is bubble sort slower than binary search? (think about comparisons).

What is Selection Sort?

- Imagine you have a box of mixed crayons (red, blue, green, yellow. . .).
- Your teacher says: **“Arrange them from smallest to largest by size.”**
- How will you do it?
 - Look at the whole box and find the **smallest crayon**.
 - Put it at the **first position**.
 - Then, look at the remaining crayons, find the **next smallest**, and put it in the second place.
 - Repeat until all crayons are in order.
- That's exactly what **Selection Sort** does!

Pass 1 and Pass 2

Initial List: [5, 3, 8, 4, 2]

Pass 1 (i = 0)

- Find min in [5, 3, 8, 4, 2] $\rightarrow$ min = 2 at index 4
- Swap with index 0 $\rightarrow$ [2, 3, 8, 4, 5]

$\Rightarrow$ End of Pass 1: [2, 3, 8, 4, 5]

Pass 2 (i = 1)

- Find min in [3, 8, 4, 5] $\rightarrow$ min = 3 at index 1
- Already in place $\rightarrow$ [2, 3, 8, 4, 5]

$\Rightarrow$ End of Pass 2: [2, 3, 8, 4, 5]

Pass 3 and Pass 4

Pass 3 ($i = 2$)

- Find min in $[8, 4, 5] \rightarrow \text{min} = 4$ at index 3
- Swap with index 2 $\rightarrow [2, 3, 4, 8, 5]$

$\Rightarrow$ End of Pass 3: $[2, 3, 4, 8, 5]$

Pass 4 ($i = 3$)

- Find min in $[8, 5] \rightarrow \text{min} = 5$ at index 4
- Swap with index 3 $\rightarrow [2, 3, 4, 5, 8]$

$\Rightarrow$ End of Pass 4: $[2, 3, 4, 5, 8]$

Pass 5 ($i = 4$)

- Only one element left $\rightarrow$ no action.

$\Rightarrow$ Final Sorted List: $[2, 3, 4, 5, 8]$

Selection Sort

Example (Syntax)

```
numbers = [64, 25, 12, 22, 11]
print("Before sorting:", numbers)

for i in range(len(numbers)):
    min_index = i

    for j in range(i+1, len(numbers)):
        if numbers[j] < numbers[min_index]:
            min_index = j

    numbers[i], numbers[min_index] =
        numbers[min_index], numbers[i]

print("After sorting:", numbers)
```

Complexity of Selection Sort

- **Best Case:** $O(n^2)$ (still scans all elements)
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Conceptual Questions

- 1 Why does Selection Sort always have a time complexity of $O(n^2)$ even if the list is already sorted?
- 2 How is Selection Sort different from Bubble Sort in terms of the number of swaps?
- 3 Is Selection Sort a stable sorting algorithm? Explain your answer.
- 4 Can Selection Sort be used efficiently on large datasets? Why or why not?

Trace / Programming Questions

- 1 Given the list [7, 2, 5, 3, 1], show the state of the list after each pass of Selection Sort.
- 2 How many swaps will Selection Sort perform on the list [4, 3, 2, 1]?
- 3 Write Python code to implement Selection Sort and print the list after each pass.
- 4 Modify Selection Sort to sort in descending order.

Real-World / Analogy Questions

- 1 You have a stack of books of different thickness. How would Selection Sort help arrange them from thinnest to thickest?
- 2 Imagine you are picking the shortest player first for a game. Explain why this is similar to Selection Sort.

Comparison Table

Feature	Bubble Sort	Selection Sort
How it moves elements	Swaps adjacent elements repeatedly	Picks smallest/largest from unsorted part
Number of swaps	Many per pass	At most one per pass
Analogy	Kids in line swapping	Picking smallest crayon from a pile
Best use	When swaps are cheap	When swaps are costly
Memory Tip	"Bubbles to the top"	"Select the smallest"

What is Insertion Sort?

- Imagine arranging books on a shelf by height.
- Start with the second book:
 - Compare it with the first book.
 - If it's smaller → shift the first book to the right and insert this one.
- Move to the third book:
 - Compare it with previous books.
 - Shift bigger ones to the right, insert in correct place.
- Repeat until all books are sorted.
- Works like inserting a card in a sorted deck!

Step-by-Step Trace of [12, 11, 13, 5, 6]

- **Initial List:** [12, 11, 13, 5, 6]
- **Pass 1 (i=1, key=11):** compare 11 with 12 → shift 12 → insert 11 → [11, 12, 13, 5, 6]
- **Pass 2 (i=2, key=13):** 13 > 12 → insert in place → [11, 12, 13, 5, 6]
- **Pass 3 (i=3, key=5):** shift 13, 12, 11 → insert 5 → [5, 11, 12, 13, 6]
- **Pass 4 (i=4, key=6):** shift 13, 12, 11 → insert 6 → [5, 6, 11, 12, 13]
- **Final Sorted List:** [5, 6, 11, 12, 13]

Insertion Sorting

Example (Syntax)

```
numbers = [12, 11, 13, 5, 6]

for i in range(1, len(numbers)):
    key = numbers[i]
    j = i - 1
    while j >= 0 and numbers[j] > key:
        numbers[j + 1] = numbers[j]
        j -= 1
    numbers[j + 1] = key

print("Sorted list:", numbers)
```

Complexity of Insertion Sort

- **Best Case:** $O(n^2)$ (still scans all elements)
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Questions

- Sort step by step: [12, 11, 13, 5, 6].
- Why is the first element always considered sorted?
- Which element will move first in: [25, 20, 30, 35, 40]?

Questions

- Sort step by step: [7, 3, 9, 2].
- If the list is already sorted: [2, 4, 6, 8, 10], how many shifts happen?
- What if the list is reverse: [10, 8, 6, 4, 2]?

Questions

- Arrange notebooks by roll numbers: [15, 10, 20, 5, 25].
- When is insertion sort fast? When is it slow? (Hint: small vs big lists, sorted vs jumbled).
- Why is insertion sort good for small lists but not for very large lists?

Questions

Without sorting fully, tell where 6 will end up after running insertion sort on: [9, 8, 7, 6, 5].

Merge Sort

- Split the pile into 2 smaller piles.
- Keep splitting until each pile has just 1 card (and a single card is always sorted).
- Then merge the piles back together, always keeping smaller numbers in front.
- That's merge sort = Divide $\rightarrow$ Conquer $\rightarrow$ Merge

Merge Sort

- 38,27,43,3,9,82,10
- 1st split [38,27,43,3] and [9,82,10]
- 2nd split [38,27], [43,3], [9,82], [10]
- 3rd split [38],[27],[43],[3],[9],[82],[10]
- 1st merge [27,38],[3,43],[9,82],[10]
- 2nd merge [3,27,38,43][9,10,82]
- 3rd merge [3,9,10,27,38,43,82]

Merge Sort - Understanding

- First: Keep breaking into smaller parts (like cutting cake slices).
- Then: Put them back in order (merge smallest $\rightarrow$ largest).
- It works step by step, and the computer is very good at repeating these steps quickly.

Merge Sorting

Example (Syntax)

```
numbers = [38, 27, 43, 3, 9, 82, 10]

parts = [[n] for n in numbers]

while len(parts) > 1:
    new_parts = []

    for i in range(0, len(parts), 2):
        if i + 1 < len(parts):
            left = parts[i]
            right = parts[i + 1]
```

Merge Sorting

Example (Syntax)

```
merged = []
while left and right:
    if left[0] < right[0]:
        merged.append(left.pop(0))
    else:
        merged.append(right.pop(0))

merged += left + right
new_parts.append(merged)
else:
    new_parts.append(parts[i])

parts = new_parts
print("Sorted:", parts[0])
```

Step-by-Step Complexity Analysis

1 Splitting

Each time, the list is divided into two halves.

For n items, we can split about $\log_2(n)$ times.

(Like folding a paper again and again).

Complexity: $O(\log n)$

2 Merging

At each level, we look at all n elements to merge back.

Complexity: $O(n)$

3 Together

Splitting $\times$ Merging = $O(\log n) \times O(n)$

$$O(n \log n)$$

Final Complexity of Merge Sort

- **Best Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$

Note: Merge Sort is more efficient than Bubble, Selection, or Insertion Sort ($O(n^2)$) for large lists.

Quick Sort

- Pick one pizza as a reference $\rightarrow$ called pivot.
- Put smaller pizzas on the left, bigger pizzas on the right.
- Repeat the same rule for left side and right side until all are sorted.

Quick Sort Example

arr = [12, 4, 7, 9, 1, 15]

- ① **Pivot = 12** → Left: [4, 7, 9, 1], Right: [15]
→ [4, 7, 9, 1] + [12] + [15]
- ② **Pivot = 4** (Left part) → Left: [1], Right: [7, 9]
→ [1] + [4] + [7, 9]
- ③ **Pivot = 7** (Sorting [7, 9]) → Left: [], Right: [9]
→ [7, 9]

Final Sorted List: [1, 4, 7, 9, 12, 15]

Quick Sorting

Example (Syntax)

```
arr = [12, 4, 7, 9, 1, 15, 20, 3]

stack = [arr]
sorted_parts = []
while stack:
    current = stack.pop()
    if len(current) <= 1:

        sorted_parts.append(current)
        continue

    pivot = current[0]
    left = []
    right = []
```

Quick Sorting

Example (Syntax)

```
for x in current[1:]:
    if x < pivot:
        left.append(x)
    else:
        right.append(x)
if right:
    stack.append(right)
stack.append([pivot])
if left:
    stack.append(left)

final_sorted = []
for part in sorted_parts[::-1]:
    final_sorted.extend(part)
print("Final Sorted:", final_sorted)
```

Quick Sort Complexity

- **Best Case:** $O(n \log n)$
Pivot splits list evenly at each step.
- **Average Case:** $O(n \log n)$
Balanced splits most of the time.
- **Worst Case:** $O(n^2)$
Pivot is always smallest/largest (like sorted list).
- **Space Complexity:**
 - Recursive calls (Best/Average): $O(\log n)$
 - Worst Case (deep stack): $O(n)$